Colin Kapsner

Computer Science Capstone

September 22, 2026

For this enhancement, I'm using the same artifact as for the first one. I selected my Android inventory management application.

I added a search feature that uses linear search to find inventory items by name within an ArrayList<InventoryItem>. I also added four sorting options that allow users to sort by name or quantity in ascending or descending order. Instead of relying on Java's sorting method, I created an InventoryAlgorithms class that implements merge sort. Linear search has a time complexity of O(n), while merge sort has a time complexity of O(n log n), making it a more efficient choice as the inventory grows.

This enhancement supports course outcomes three and four by demonstrating my ability to select and implement appropriate algorithms and data structures. I used an ArrayList to manage inventory objects and implemented searching and sorting algorithms to solve a practical problem within the app. I also separated these algorithms from the user interface, again improving the organization.

I used this enhancement to make myself understand the difference between using an existing algorithm and implementing one myself. Replacing Java's built-in sorting with merge sort required me to understand how the algorithm divides and merges data (GeeksforGeeks, 2026). Implementing linear search also gave me a practical example of O(n) complexity.

Below is a checklist of enhancements I made while working on this enhancement:
Created Functional Search Box

- Added an EditText search field (@+id/editSearch) underneath the Add Item button.
- Added a TextWatcher to dynamically update results as the user types.
- Search operates on the in-memory inventory collection rather than repeatedly querying the database.

Added Sort Options to Overflow Menu (using my custom sorting algorithm)

- Name A-Z
- Name Z-A
- Quantity Low-High
- Quantity High-Low
- Added selection handling to update the active sort option and refresh the displayed inventory.

Implemented Centralized Algorithms

- Added searchByName(...) using a case-insensitive linear search.
- Implemented custom Merge Sort algorithms for all four sorting options.
- Refactored InventoryActivity so searching and sorting are handled by InventoryAlgorithms instead of UI code.

Used ArrayList<InventoryItem> as the Primary Data Structure

- Inventory records are loaded into an ArrayList.
- Search and sorting algorithms operate directly on the collection of InventoryItem objects.
- Results are passed to the RecyclerView adapter for display.

Analyzed Algorithm Efficiency

- Linear search has O(n) time complexity.
- Merge sort has O(n log n) time complexity.
- Selected merge sort instead of a simpler $O(n^2)$ algorithm to provide better scalability as inventory size increases.

Reference

GeeksforGeeks. (2026, August 6). *Merge sort*. https://www.geeksforgeeks.org/dsa/merge-sort/